AI 101 for Mechanical Engineers · Module 11

A neuron is a small computation

Lesson 11.1

What you will learn

> Explain the roles of inputs, weights, bias and activation in a neuron, calculate a ReLU output, and explain why layers need nonlinear operations between them.

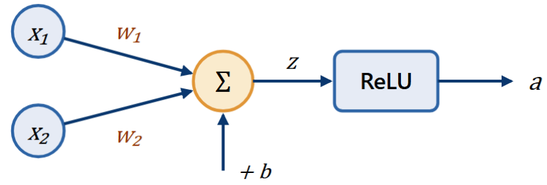
Lesson overview

Module 11 · Lesson 1

A neuron is a small computation

AI 101 for Mechanical Engineers

1 Follow the computation



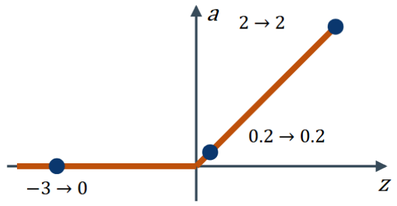
$$z = w_1x_1 + w_2x_2 + b, \quad a = \max(0, z)$$

$x_1 = 2$, $x_2 = 1$, $w_1 = 0.5$, $w_2 = -1$, $b = 0.2$

$w_1x_1 = 0.5 \times 2$	**= 1**
$w_2x_2 = -1 \times 1$	**= −1**
$z = 1 + (-1) + 0.2$	**= 0.2**
$a = \max(0, 0.2)$	**= 0.2**

Weights and bias are fitted parameters. A neuron and a layer describe the structure of the computation.

2 ReLU: a = max(0, z)

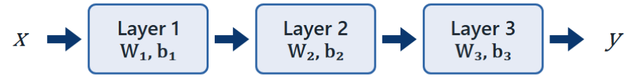


z	a = max(0, z)
−3	0
0.2	0.2
2	2

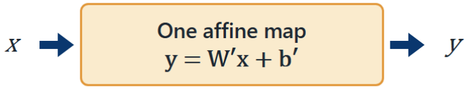
Negative values become 0; positive values pass unchanged. Sigmoid and tanh are other activations.

3 Layers without nonlinearity collapse

Three affine layers applied in series:



=



$$W' = W_3W_2W_1$$
$$b' = W_3W_2b_1 + W_3b_2 + b_3$$

The map is **affine**; it is linear only when $b' = 0$. A nonlinear activation such as ReLU between layers prevents this collapse.

Units: all values in this teaching example are dimensionless. With physical inputs, state the units and keep each weighted term and the bias in consistent units.

Key takeaway

A neuron computes a weighted sum plus bias, then an activation.
Without nonlinear activations, any number of stacked affine layers is still one affine map.

AI 101 for Mechanical Engineers · Lesson 11.1

© 2026 Prof. Mazharul Islam · CC BY 4.0

A neuron: weighted sum, bias, then activation

$$z = \Sigma_i \, w_i x_i + b$$

$$z = w_1 x_1 + w_2 x_2 + b$$

$$a = f(z)$$

- An artificial neuron is a mathematical computation [4]
- x_i input, w_i weight, b bias, z value before activation
- Example values are dimensionless; physical inputs need stated, consistent units

Parameters are fitted; structure is arranged

Fitted parameters

- Weight: learned multiplier scaling an input
- Bias: additive term shifting the weighted sum

Structure

- Neuron: performs a computation
- Layer: a collection or stage of computations
- Network: combines layers into a mapping

> Here “bias” is the additive term, not statistical or fairness bias.

Identical starting weights are a problem

- Deterministic training gives identical neurons the same outputs and gradients
- They then have no reason to learn different features
- Common initializers use small, varied values to break symmetry

  [4, Sec. 8.4]

- Initialization does not guarantee a useful representation

ReLU keeps positive values and zeroes negative ones

$$a = \max(0, z)$$

- Rectified Linear Unit: select the larger of 0 and z [60]
- Negative z gives 0; z = 0 gives 0; positive z passes unchanged
- This piecewise behaviour introduces nonlinearity

Three ReLU examples

z	a = max(0, z)
−3	0
0.2	0.2
2	2

All values are dimensionless. Check which branch applies before further calculation.

Worked example: the contributions cancel

1. First contribution: $w_1x_1 = 0.5 \times 2 = 1$
2. Second contribution: $w_2x_2 = -1 \times 1 = -1$
3. Add bias: $z = 1 + (-1) + 0.2 = 0.2$
4. Activate: $a = \max(0, 0.2) = 0.2$
5. Forward calculation only: no gradient, no weight change

Stacked affine layers collapse into one

$$W_3(W_2(W_1x + b_1) + b_2) + b_3 = W'x + b'$$
$$W' = W_3W_2W_1$$
$$b' = W_3W_2b_1 + W_3b_2 + b_3$$

- Linear maps without nonlinear operations stay linear [4, Sec. 6.1]
- With bias terms the result is affine; linear only if $b' = 0$
- ReLU between layers prevents this collapse; accuracy is not guaranteed

ReLU is one activation among several

ReLU

- a = max(0, z)
- Used here for its simple calculation

Sigmoid and tanh

- Sigmoid: interval 0 to 1 [4, Sec. 6.3.2]
- Often for a binary-classification output
- tanh: interval −1 to 1 [4, Sec. 6.3.2]

ReLU is not best for every layer or task.

CHECK YOUR UNDERSTANDING

What are the outputs of ReLU for $z = -3$ and $z = 2$?

Answer before moving on.

CHECK YOUR UNDERSTANDING

What are the outputs of ReLU for $z = -3$ and $z = 2$?

> 0 and 2, respectively. ReLU selects the larger of zero and z. These values are dimensionless.

CHECK YOUR UNDERSTANDING

Why is calculating z = 0.2 in the worked example different from training the neuron?

Answer before moving on.

CHECK YOUR UNDERSTANDING

Why is calculating z = 0.2 in the worked example different from training the neuron?

> It uses specified inputs, weights and bias to produce an output. Training also needs an objective, gradient calculation and a parameter update.

Key takeaways

- A neuron computes a weighted sum plus bias, then activation
- Weights and bias are fitted; neurons and layers are structure
- ReLU maps negative values to zero, leaves positive unchanged
- Stacked affine layers stay affine without nonlinear operations

References

[4] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[60] V. Nair and G. E. Hinton, “Rectified linear units improve restricted Boltzmann machines,” in Proc. 27th Int. Conf. Mach. Learn. (ICML), 2010, pp. 807–814.

Numbers match the course reference list. The written lesson is the authoritative text.